

QCon
全球软件开发大会

从单点辅助到 Agent 闭环： 基于 Agent Skills、MCP 与 Playwright 的 全链路智能化测试实践

蔡明哲

Ubiquiti
Quality Assurance

极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

🕒 6月 📍 上海

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：6月26-27日

- AI Infra 系统工程
- 多 Agent 协作与实践
- 多模态融合
- 模型训练与推理创新
- 数据平台与特征服务

🕒 8月 📍 深圳

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：8月21-22日

- Agentic AI
- 轻量化与高效推理
- 多模态应用
- AI + IoT 场景实践
- AI 工业化落地

🕒 10月 📍 上海

👥 1200人

QCon

全球软件开发大会

会议时间：10月22-24日

- AI Agent
- Vibe Coding
- 智能可观测
- 推理基建
- 模型攻防
- AI x 创造力

🕒 12月 📍 北京

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：12月18-19日

- 大模型架构创新
- 多模态 AI 产业融合
- 具身智能
- AI for Science
- 大模型安全



蔡明哲 (Max)



Ubiquiti (优倍快) Quality Assurance

深耕测试与品质工程领域，专注于测试效能提升、自动化测试架构与 AI 驱动测试方法。曾于多场技术研讨会担任讲者，包括中国互联网测试开发大会、DevOpsDays、Test Corner 等。著有 UI 自动化测试与 AI 应用实战一书，并制作软体测试相关线上课程，在实务上，曾设计测试架构与方法，使回归测试时间大幅缩短最高达 95%，并参与 LLM 与 Multi Agent 系统品质体系建置，持续探索如何以工程化、自动化与智能化手段，全面提升产品品质与测试效率。

目录

01 背景与挑战

02 工程实践

03 核心技术底座

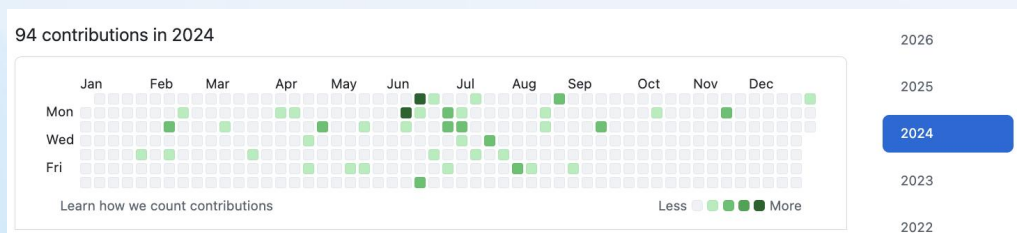
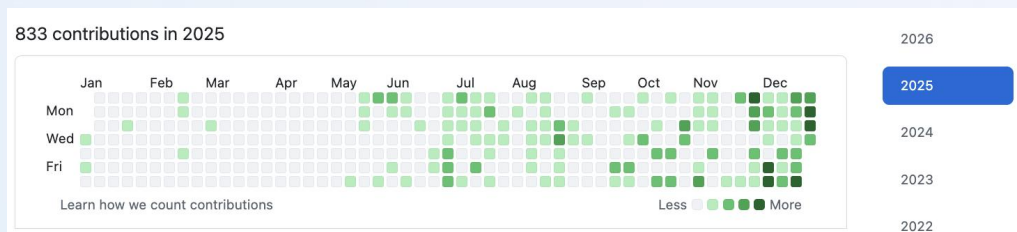
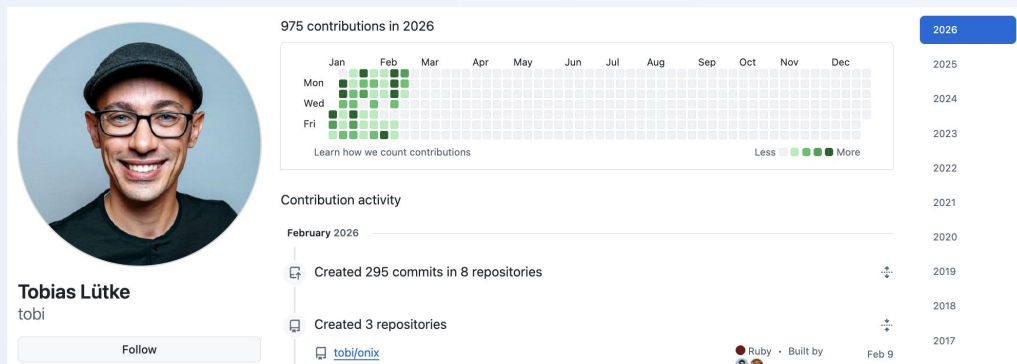
04 未来展望

01 背景与挑战



AI 对 QA 是帮助比较多？
还是威胁比较多？

AI 加速研发：别让 QA 成为 AI 时代的瓶颈



Shopify CEO 历年 GitHub Contributions

随着 AI 赋能开发端实现显著提速，若 QA 环节未能同步进化，测试端将不可避免地成为专案交付的效能瓶颈。因此，QA 体系也必须深度整合大模型力量，从自动化脚本生成、边界案例预测到异常分析进行全面优化，确保『测试速率』能与『开发速率』并驾齐驱，实现端对端的研发效能转型。

1

需求堆积： 待测功能在 QA 阶段形成严重排队。

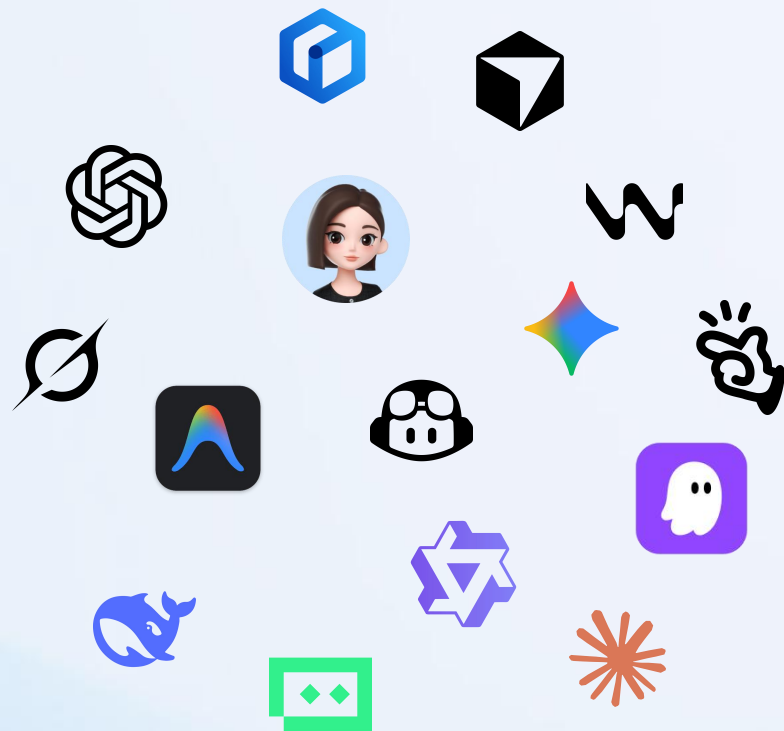
2

品质风险： 为了赶上发布时间而压缩测试覆盖率。

3

协作断层： 开发已进入下个冲刺，QA 还在处理上个版本的 Bug。

工具链的「破碎化」与「孤岛现象」



人机分工模糊不清

团队未清晰界定 AI 与人工的职责边界，常让 AI 承担复杂业务逻辑设计、缺陷深度根因分析等高阶认知任务，同时却让让人工长期重复执行可被自动化的基础工作，造成双方能力错配与资源浪费。

对 AI 产出信任失衡

过度依赖 AI，将其视为万能解决方案，直接使用生成的测试用例却不做业务校验，导致场景脱离实际；或是因不信任 AI 准确性而完全排斥使用，错失提升测试效率与覆盖率的机会。

工具孤岛内耗

受工具孤岛影响，QA 需要在测试管理、缺陷平台、AI 工具、需求文档等多个系统间频繁切换，手动复制数据、补全信息、对齐上下文，大量精力消耗在非核心的机械性工作中。

● 认知偏差：将 AI 视为工具而非协同伙伴，忽视人机配合价值

你是一位资深的QA工程师，
擅长撰写测试用例，
请帮我撰写一份关于xxx功能的测试

一句简短单一的指令



生成低质量、低可用性测试用例



放弃使用

从入门到放弃

上下文断裂，知識局限在個人

许多 QA 对于 AI 的使用仅仅使用 ChatGPT 等网页的聊天介面，上下文永远断裂，无法累积业务背景与测试脉络，改一次需求就要重来一遍 prompt

工具效能未释放，业务任务错配

未结合业务领域、测试规范与技术栈做深度适配与能力增强，缺少针对性配置、prompt 工程优化与场景化微调，导致 AI 生成内容通用性强、业务贴合度弱，难以真正发挥自动化生成、智能分析与决策辅助的最大价值。

业务知识无沉淀，无法复用

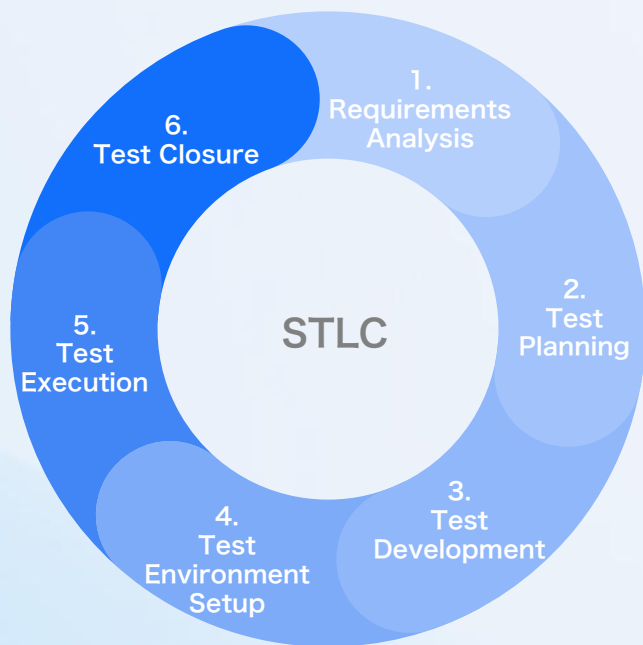
业务知识、测试上下文难以与 AI 有效结合，适配业务的提示词、AI 使用经验仅停留在个人层级，未系统化沉淀为组织资产，无法在团队内复用于同类业务测试。

02 工程实践

从案例生成到自动修复的全流程落地

QA 日常工作内容

QA 核心工作围绕 STLC 与日常任务展开，传统模式依赖人工执行效率受限，通过在需求分析、用例设计、脚本开发、执行调试、缺陷管理全流程植入 AI 能力，可全面自动化、智能化升级测试工作。



AI 赋能测试革新：从单点突破到全域提速

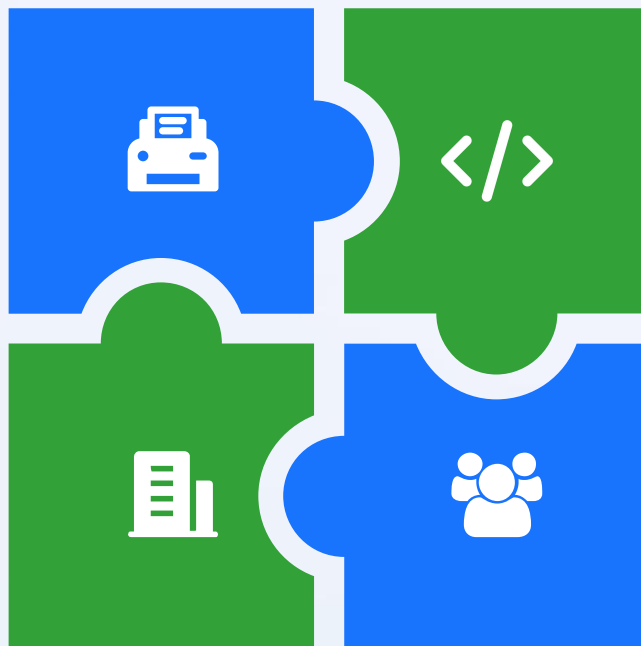
以智能技术重构测试全流程，实现从单点优化到全链路提效的跨越

测试用例生成

基于 AI 的分析能力，快速解析需求文档与接口定义，透过结构提示词，精准生成测试用例，大幅缩短用例设计周期，摆脱人工编写的低效重复。

产品业务知识提取

通过大模型处理与知识图谱技术，从产品文档、历史项目中高效提取核心业务规则与逻辑关联，快速构建测试知识底座，减少业务理解成本及作为测试案例设计参考文档。



测试脚本生成

AI Agent 自主理解测试场景，自动生成可执行脚本并完成自动化测试执行，支持多场景适配，降低脚本编写门槛，提升测试执行效率与稳定性。

用户反馈 Log 分析

AI 智能解析海量用户反馈与日志，自动定位高频问题、异常场景及潜在风险点，将零散数据转化为可落地的测试重点。

AI 赋能测试效能提升：四大核心能力量化实践成果

测试用例生成 60%

依托 AI 深度解析需求文档与风险变更，结合结构化提示词精准输出高覆盖测试用例，用例设计周期缩短 60%，用例覆盖率与合规性显著提升。

产品业务知识提取 75%

通过大模型与知识图谱技术，从产品文档、历史项目中提取核心业务规则与逻辑关系，快速构建测试知识底座，业务理解成本降低 70%，知识沉淀与文档输出效率提升 75%。

测试脚本生成 65%

AI Agent 自主理解测试场景并自动生成可运行脚本，完成全流程自动化执行，脚本编写效率提升 65%，大幅降低人工编码成本与维护成本。

用户反馈 Log 分析 70%

AI 智能解析海量日志与用户反馈，自动识别高频问题、异常场景与潜在风险，问题定位效率提升 70%，风险前置识别率提升 60%，为测试重点与质量优化提供精准依据。

四大核心能力量化成效与业务价值

AI 赋能测试革新：从点效突破到全域提速

以智能技术重构测试全流程，实现从单点优化到全链路提效的跨越

测试用例生成

基于 AI 的分析能力，快速解析需求文档与接口定义，透过结构提示词，精准生成测试用例，大幅缩短用例设计周期，摆脱人工编写的低效重复。

产品业务知识提取

通过大模型处理与知识图谱技术，从产品文档、历史项目中高效提取核心业务规则与逻辑关联，快速构建测试知识底座，减少业务理解成本及作为测试案例设计参考文档。



测试脚本生成

AI Agent 自主理解测试场景，自动生成可执行脚本并完成自动化测试执行，支持多场景适配，降低脚本编写门槛，提升测试执行效率与稳定性。

用户反馈 Log 分析

AI 智能解析海量用户反馈与日志，自动定位高频问题、异常场景及潜在风险点，将零散数据转化为可落地的测试重点。

Agent Skills 落地测试案例生成：核心应用与实操技巧

Agent Skills 赋能 AI 自动化生成测试用例，核心是将大模型调适为贴合业务的测试专家助理，通过投喂历史测试数据、需求文档与代码库，让 AI 自主识别功能边界、用户行为路径与异常场景组合，大幅提升用例生成效率；落地过程中掌握关键实操技巧，能进一步保障生成用例的可用性与稳定性，让 Agent Skills 的能力充分发挥。

```
---
name: test-cases
description: This document describes a
process for generating structured QA test
cases from product requirements or feature
specifications.
```

Objective

Transform product requirements into
organized and actionable test cases that
validate:

- Core functionality
- Edge and boundary conditions
- Failure scenarios and error handling
- Workflow or state transitions

The goal is to ensure each requirement has
sufficient testing coverage and can be verified
by QA.

.....

收集需求，如产品需求文档



识别测试场景



定义测试用例结构



生成测试用例

1

QA 基础认知：QA 需理解 AI 生成用例核心逻辑，明确 AI 可自主识别功能边界条件、用户行为路径、异常场景组合等三类核心测试场景，精准把控 AI 能力边界与应用方向

2

拆分测试类型：摒弃 all in one 的生成思路，按压力测试、功能测试、PICT 测试案例等不同测试类型独立配置 Agent Skills，适配各类型测试的专属需求

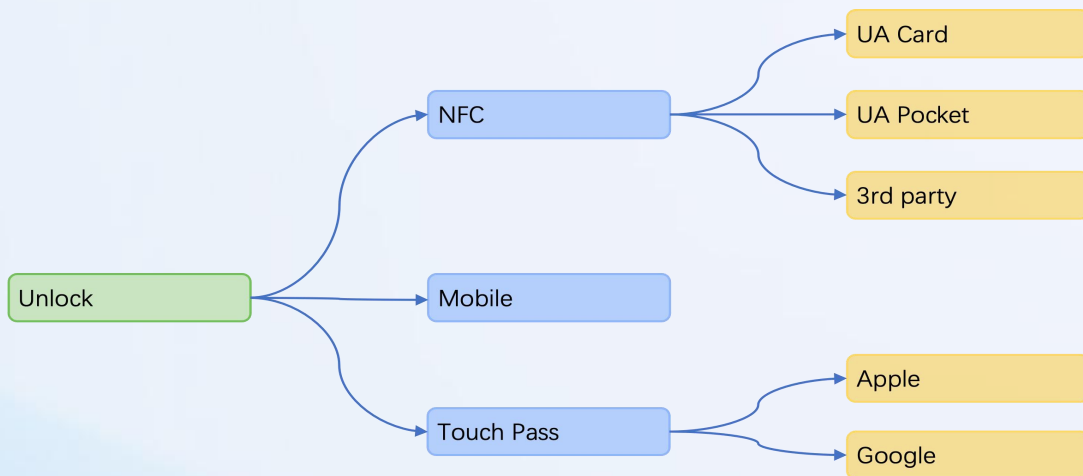
3

规范输入结构：以结构化的方式向 Agent Skills 提供需求、业务规则等信息，结构化输入是 AI 生成可用、稳定测试用例的关键前提

AI 驱动测试点生成：先点后案的敏捷测试策略

Test Case ID	Test Description	Test Env	Preconditions	Test Steps	Expected Result	Status
1	開啟 NFC 開門後開門成功	All	錄製NFC卡片並分配權限	1. 開啟設備設定頁面 2. 點擊 NFC 功能開啟 3. 按下確定儲存 4. 刷卡	刷卡開門成功	Pass
2	關閉 NFC 開門後開門失敗	All	錄製NFC卡片並分配權限	1. 開啟設備設定頁面 2. 點擊 NFC 功能關閉 3. 按下確定儲存 4. 刷卡	刷卡開門失敗，聲音播報錯誤提示	Pass

完整测试案例示意图



心智图测试案例示意图

受限于业务逻辑认知不足，大模型难以生成完整的测试步骤，但透过高质量的上下能，大模型也能高效产出「测试点」，可作为测试案例产出或是完善测试点，面对高速迭代，我们采取「先点后案」策略：先由 AI 快速生成测试点以匹配开发节奏，待发布后再将高价值场景转化为回归用例，兼顾灵活响应与资产沉淀。



大模型升级测试点，人工检阅后采用，并在迭代稳定后补齐完整测试案例内容



Agent Skills 案例 - PICT 成对组合测试

依規格擷取可組合的參數與選項，建立 Pairwise 模型。

```
# PIN Unlock 功能 - 參數與選值
PIN_Length: 4_Digits, 6_Digits, 8_Digits, Variable_Length
Keypad_Layout: Standard, Randomized
PIN_Assignment: Custom_Entered, Random_Generated
Reader_Type: Intercom, Reader_Pro
```

參數說明：

- PIN_Length**：全讀卡機的 PIN 長度設定（4/6/8 位固定或可變）。
- Keypad_Layout**：單一讀卡機的鍵盤版面（標準 / 隨機）。
- PIN_Assignment**：指派 PIN 時為手動輸入或系統隨機產生。
- Reader_Type**：讀卡機類型（鍵盤版面設定適用 Intercom 與 Reader Pro）。

約束：規格未限定上述四者之間的互斥條件，故無 PICT 約束；僅以 Pairwise 覆蓋組合。

三、Generated Test Cases (Pairwise)

Test #	PIN_Length	Keypad_Layout	PIN_Assignment	Reader_Type	Expected Output
1	4_Digits	Standard	Custom_Entered	Intercom	可設定 4 位 PIN；標準鍵盤；手動輸入 PIN 成功；Intercom 上可正常解鎖。
2	4_Digits	Randomized	Random_Generated	Reader_Pro	可設定 4 位 PIN；隨機鍵盤；隨機產生 PIN 成功；Reader Pro 上可正常解鎖。
3	6_Digits	Standard	Random_Generated	Reader_Pro	可設定 6 位 PIN；標準鍵盤；隨機產生 PIN 成功；Reader Pro 上可正常解鎖。
4	6_Digits	Randomized	Custom_Entered	Intercom	可設定 6 位 PIN；隨機鍵盤；手動輸入 PIN 成功；Intercom 上可正常解鎖。
5	8_Digits	Standard	Custom_Entered	Intercom	可設定 8 位 PIN；標準鍵盤；手動輸入 PIN 成功；Intercom 上可正常解鎖。

幫我閱讀 PIN code 的新功能規格後，創建測試案例

從 references/Configuring-PIN-Unlock-in-UniFi-Access.md 等整理出：

- 指派 PIN：Settings > Admins & Users > Users → 使用者 → Assignments → Add PIN（可手動輸入或隨機產生）。
- PIN 長度：Settings > General > PIN → 固定 4/6/8 位或可變長度（套用所有讀卡機）。
- 鍵盤版面：Devices > Readers > [Intercom/Reader Pro] > Access Methods > PIN > Keypad Layout → Standard 或 Randomized。
- 開門流程：讀卡機上點 PIN Unlock → 輸入 PIN → Enter → 解鎖。
- 業務規則：每位使用者僅能有一個 PIN。

2. 測試案例 (PICT Test Designer)

依規格抽出 4 個參數並做 Pairwise 組合：

參數	選值
PIN_Length	4_Digits, 6_Digits, 8_Digits, Variable_Length
Keypad_Layout	Standard, Randomized
PIN_Assignment	Custom_Entered, Random_Generated
Reader_Type	Intercom, Reader_Pro

- PICT 模型：無額外約束（規格未定義參數互斥）。
- 產出：8 條 Pairwise 測試案例，每條都有預期結果說明。
- 補充：文件內另建議「一使用者一 PIN」「錯誤 PIN」「訪客 PIN」等單點驗證。

PICT Agent Skills 实际运用

透过将微软 PICT 算法封装为 Agent Skills，赋予 AI 极致的测试减法能力。当面对 8 参数系统，PICT 能将 19,440 个穷举案例压缩至仅 30 个（减少 99.88%），同时完整覆盖所有成对交互风险。透过此技能，Agent 可直接从自然语言需求提取参数、应用约束逻辑，并输出最优测试集，以最小成本实现高覆盖率的测试。

交易类型 (5)：提款、存款、余额查询、转帐、变更 PIN 码

卡片类型 (3)：EMV 晶片卡、磁条卡、无效卡

PIN 状态 (4)：有效、无效（尝试 1 - 3 次）

帐户类型 (3)：支票帐户、储蓄帐户、两者皆有

交易金额 (4)：在限额内、达到最大限额、超过单笔交易限额、超过每日限额

现金可用性 (3)：充足、不足、已用尽

网路状态 (3)：主要网路、备援网路、已断线

卡片状态 (3)：正常、损坏、已过期

可能的总组合高达数：19,440

Agent Skills 案例 - 基于代码变更的智能测试

透过分析代码变更自动产出结构化测试计画，核心价值在于只测改动风险点，而非全量执行。整体流程透过 GitHub MCP 解析变更资讯，分析程式影响范围与风险等级，结合 BM25、Cosine Similarity + HNSW 与 RRF，从 TestRail 与产品知识库精准匹配测试案例，并完成测试缺口分析与案例补充，实现精准、高效、风险导向的智能测试规划。





MCP 驱动 TestRail 的集成与智能编排

核心增益

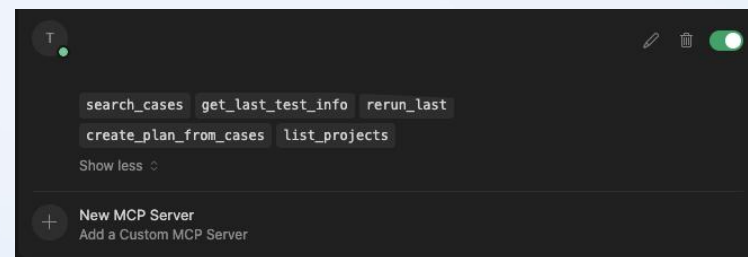
- **历史用例复用**: 查询上一版测试案例，自动在 TestRail 创建新 test run，提升用例复用效率。
- **智能测试建议**: 基于当前 TestRail 中的 test case，结合业务场景输出针对性测试执行建议。
- **进度数据统整**: 提取 TestRail 中的测试进度数据，可嵌入标准化测试报告。

落地挑战

- **多项目管理难题**: 企业内 Project 数量过多时，易出现范围混乱，需制定专属规则或限定操作范围。
- **个性化适配需求**: 企业内部 test plan 架构、操作流程多为定制化，开源 MCP 工具难以完全匹配，定制化 MCP 适配性更优。
- **权限管控挑战**: 多团队共用 TestRail 时，MCP 整合需精细化权限设计，避免跨项目数据操作风险。



将 MCP 与主流测试管理工具 TestRail 完成技术整合，打通测试管理与自动化执行的链路，依托 MCP 的拓展能力实现 TestRail 上测试用例、测试计划的自动化操作与数据联动，同时为测试执行提供专业建议、为测试报告沉淀核心进度数据，大幅减少测试管理的人工操作成本；但在企业级落地过程中，也需应对多项目管理、个性化架构适配等实际问题，需针对性优化解决方案。



Skills 构建的核心准则：领域知识为基，高效复用为果

```
.
├── agents
│   ├── analyzer.md
│   ├── comparator.md
│   └── grader.md
├── assets
│   └── eval_review.html
├── eval-viewer
│   ├── generate_review.py
│   └── viewer.html
├── LICENSE.txt
├── references
│   └── schemas.md
├── scripts
│   ├── __init__.py
│   ├── aggregate_benchmark.py
│   ├── generate_report.py
│   ├── improve_description.py
│   ├── package_skill.py
│   ├── quick_validate.py
│   ├── run_eval.py
│   ├── run_loop.py
│   └── utils.py
└── SKILL.md
```

使用 Skill-Creator 构建 Skills 是高效、可复用的最佳实践，但必须建立在扎实的领域知识之上。只有先具备完整的测试方法论、验证逻辑与质量标准，才能设计出稳定、可靠、可被 Agent 正确执行的 Skills。

- 1 逻辑确定化：**关键验证逻辑改由 Python/Bash 脚本执行，Skills 仅负责调度。彻底消除模型幻觉，确保测试结果具备工业级的准确性与可重复性。
- 2 显性激励约束：**在 Prompt 中植入「品质优于速度」等强指令，矫正模型走捷径的倾向，强制其执行深度且严谨的测试检查。
- 3 Header 权重优化：**将不可违反的守则置于 SKILL.md 顶部并标注 `## CRITICAL`，利用注意力机制确保核心规则在长上下文中始终最高权重。
- 4 像团队一样分工：**在创建 agent skills 不需要建造 all-in-one 的超级 agent，而是像 QA 团队一样分工各司其职

AI 赋能测试革新：从点效突破到全域提速

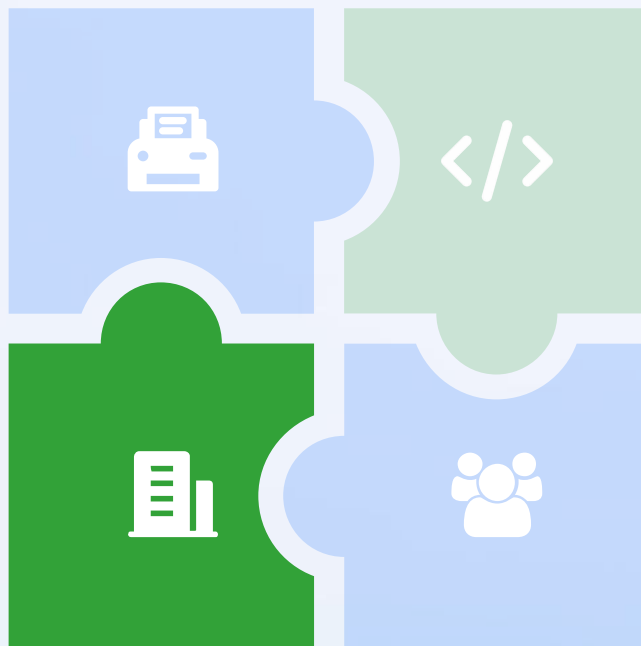
以智能技术重构测试全流程，实现从单点优化到全链路提效的跨越

测试用例生成

基于 AI 的分析能力，快速解析需求文档与接口定义，透过结构提示词，精准生成测试用例，大幅缩短用例设计周期，摆脱人工编写的低效重复。

产品业务知识提取

通过大模型处理与知识图谱技术，从产品文档、历史项目中高效提取核心业务规则与逻辑关联，快速构建测试知识底座，减少业务理解成本及作为测试案例设计参考文档。



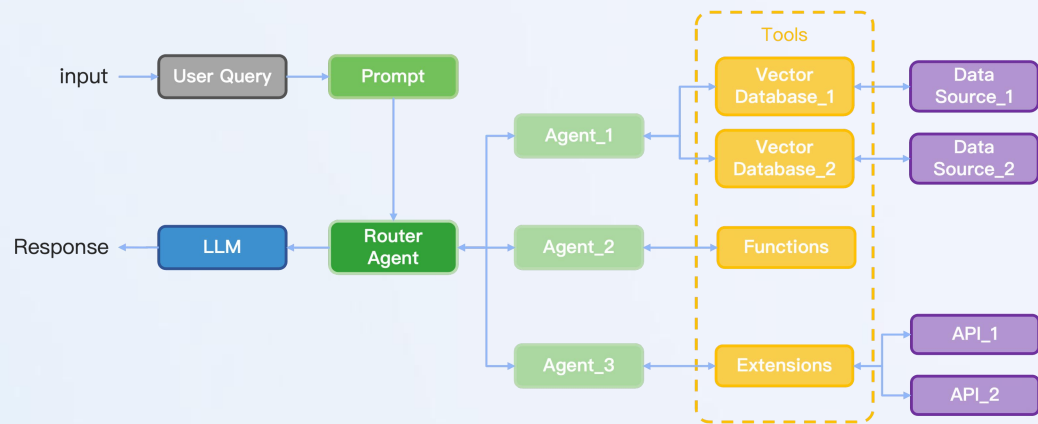
测试脚本生成

AI Agent 自主理解测试场景，自动生成可执行脚本并完成自动化测试执行，支持多场景适配，降低脚本编写门槛，提升测试执行效率与稳定性。

用户反馈 Log 分析

AI 智能解析海量用户反馈与日志，自动定位高频问题、异常场景及潜在风险点，将零散数据转化为可落地的测试重点。

测试场景下 RAG 的应用痛点：效率与适配的双重瓶颈



透过代码建立 Multi Agent RAG systems



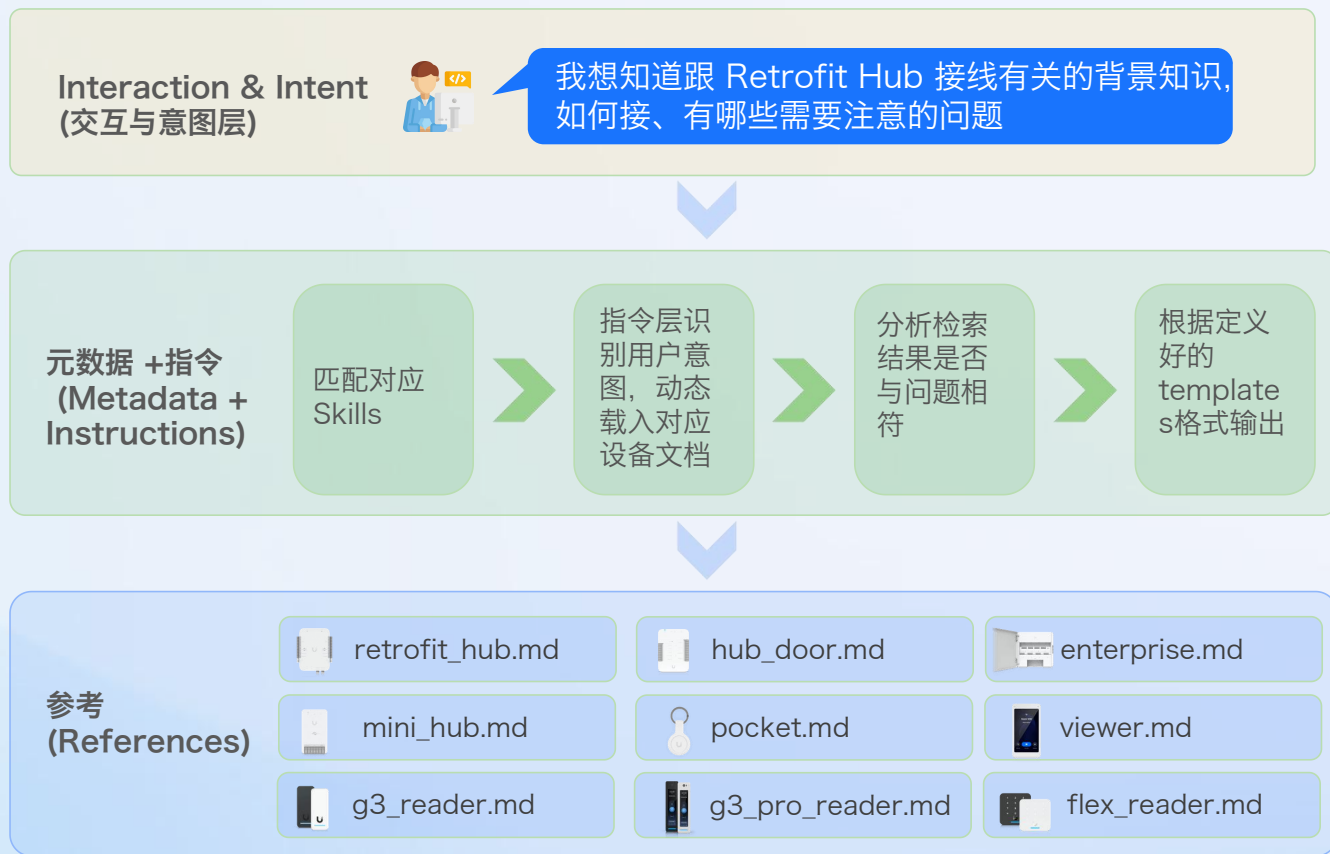
透过 Dify 建置知识库示意图

过去 RAG 方案在测试提效中的落地难题

当前 RAG 在测试领域的落地瓶颈，其中一点在投入建置完整工具的成本过高，通用工具难以处理高度异构的测试数据与严密的业务逻辑，Top K 影响最终输出结果，这会导致最终的结果难以作为实际测试场景中使用。

- 1 技术落地门槛高：** 需专业人员做数据清洗、向量库调优、检索策略配置，测试团队自主落地成本高、周期长。
- 2 场景适配性弱：** 对测试用例、业务知识等专属场景的语义理解不足，检索结果精准度低，难以直接支撑测试工作；
- 3 知识更新成本高：** 产品业务、测试需求迭代后，需重新做数据入库与向量更新，无法实现实时、自动化的知识同步。

Agent Skills: 替代传统 RAG, 打造测试专属智能能力



传统 RAG 存在建置门槛高、落地难度大的问题, Agent Skills 作为高效替代方案, 可通过自定义规范定义参考文件并完成技能初步分级, 既大幅降低建置难度, 同时更贴合业务实际执行需求, 成为解决传统 RAG 痛点的高效路径。

- 1 建置更简易:** 无需复杂的向量库搭建、高质量资料萃取、检索调优等操作, 相较传统 RAG 的繁琐流程, Agent Skills 落地门槛更低、实施更高效
- 2 规范可自定义:** 技能的参考文件、分级标准均由业务侧自主定义, 可精准匹配实际业务场景, 让技能落地更具针对性

AI 赋能测试革新：从点效突破到全域提速

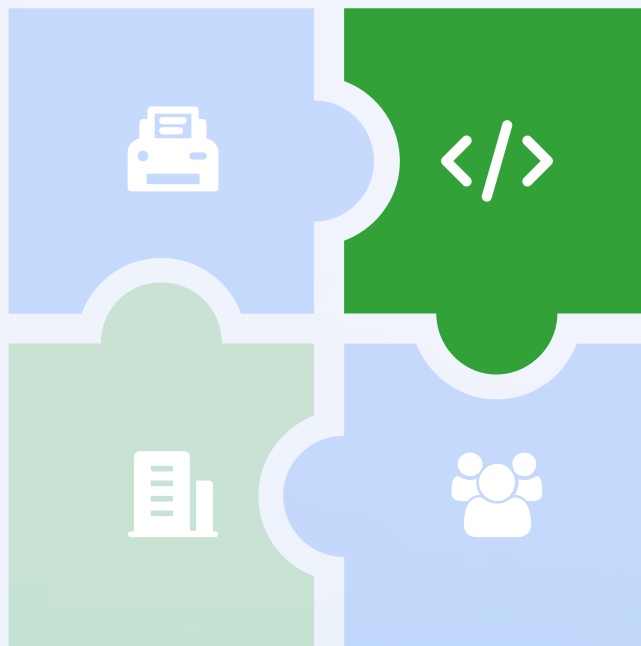
以智能技术重构测试全流程，实现从单点优化到全链路提效的跨越

测试用例生成

基于 AI 的分析能力，快速解析需求文档与接口定义，透过结构提示词，精准生成测试用例，大幅缩短用例设计周期，摆脱人工编写的低效重复。

产品业务知识提取

通过大模型处理与知识图谱技术，从产品文档、历史项目中高效提取核心业务规则与逻辑关联，快速构建测试知识底座，减少业务理解成本及作为测试案例设计参考文档。



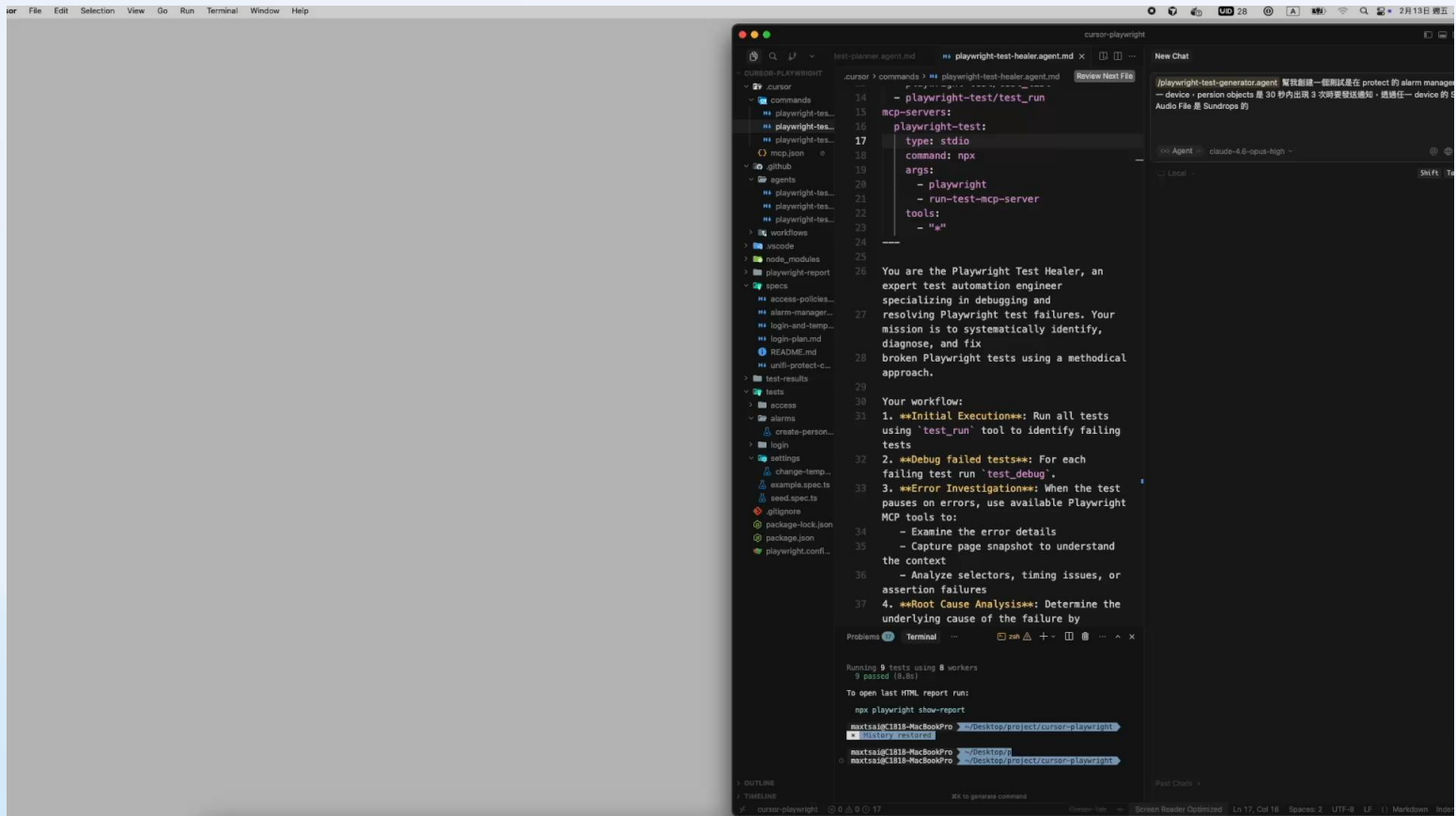
测试脚本生成

AI Agent 自主理解测试场景，自动生成可执行脚本并完成自动化测试执行，支持多场景适配，降低脚本编写门槛，提升测试执行效率与稳定性。

用户反馈 Log 分析

AI 智能解析海量用户反馈与日志，自动定位高频问题、异常场景及潜在风险点，将零散数据转化为可落地的测试重点。

Agent 执行测试并生成脚本



Playwright Agent Planner/Generator/Healer loop

在 protect 的 alarm manager 中, 任一 device, person objects 是 30 秒内出现 3 次时发送通知, 设置任一 device 的 Sound 的 Audio File 是 Sundrops

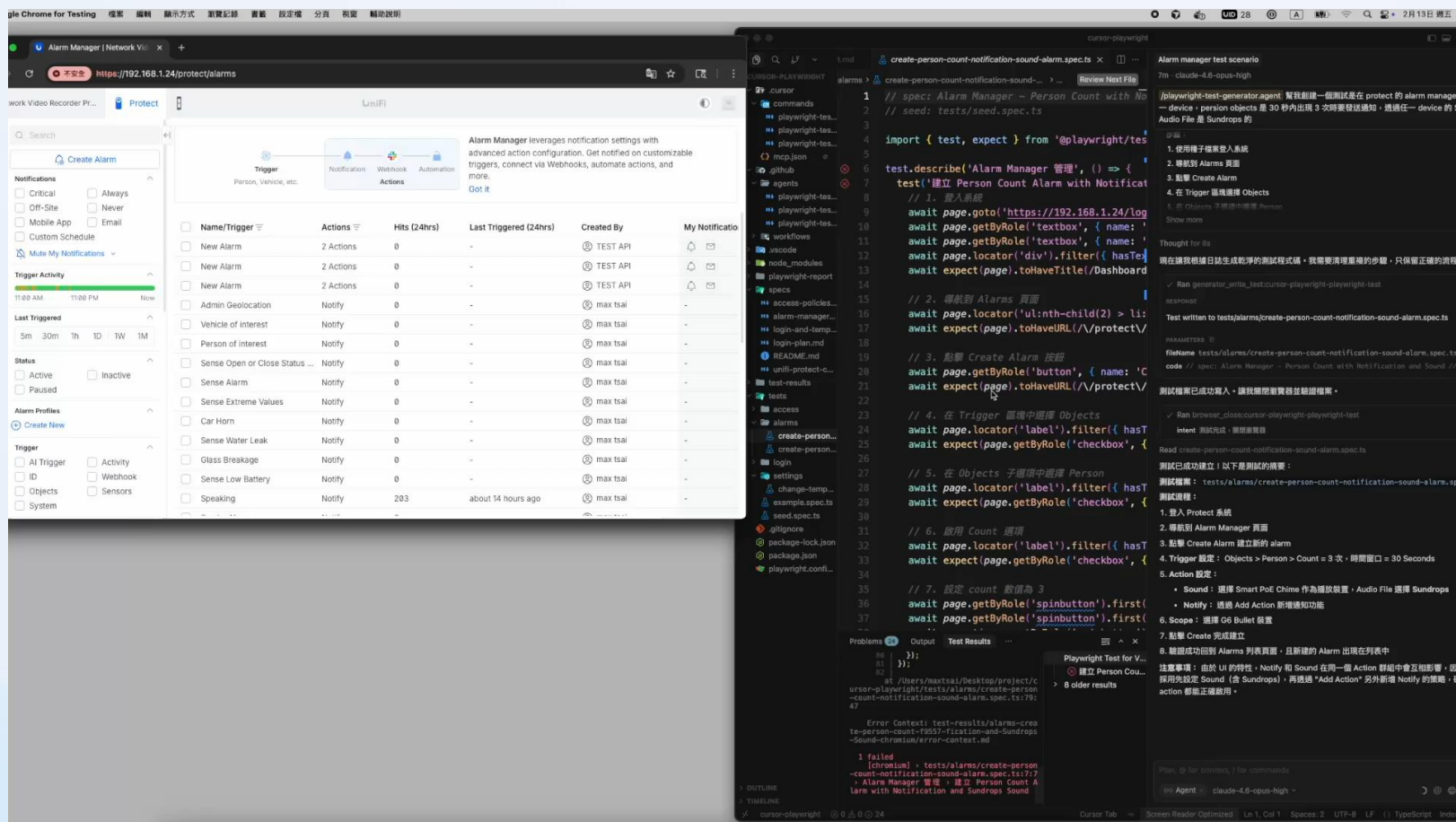


1. 点击进入 Protect Application
2. 点击进入 alarms 页面
3. 点击 Create Alarm
4. 选择人物的 Objects
5. 勾选 Person
6. 勾选 Count
7. 设定 count exceeds 是 30
8. Within 选择 Custom
9. 设定时间为 30 秒
10. 选择任一 camera
11. 选择 Sound
12. 选择 Play On
13. 选择 Audio Sound
14. 点击新增



透过探索页面生成测试案例, 能够处理需要展开后取得资讯的复杂操作

Healer Agent: 自动化测试维护的智能修复引擎



Healer Agent 智能修复

Healer Agent 针对大型测试套件的核心维护痛点设计，打破「测试失败即手动修复」的传统模式。面对 UI 结构变更、Flaky 行为、时序异常等常见问题，它以「智能检测」的角色实现从错误诊断到修复验证的全流程自动化，同时通过人机协作保障测试意图不偏离，大幅降低资深工程师的重复性劳动。



UI 自适应执行与 API 测试同源生成

```
.playwright-mcp > network-baseline.txt
1 [GET] https://192.168.1.24/api/users/self => [401]
2 [GET] https://192.168.1.24/api/system => [200]
3 [GET] https://192.168.1.24/favicon.svg?v3 => [200]
4 [GET] https://192.168.1.24/favicon.ico?v3 => [200]
5 [POST] https://192.168.1.24/api/auth/login => [200]
6 [GET] https://192.168.1.24/api/users/self => [401]
7 [GET] https://192.168.1.24/api/users/self => [200]
8 [GET] https://192.168.1.24/favicon.svg?v3 => [200]
9 [GET] https://192.168.1.24/favicon.ico?v3 => [200]
10 [GET] https://192.168.1.24/api/firmware/update => [200]
11 [GET] https://192.168.1.24/proxy/users/api/v2/user/self => [200]
12 [GET] https://192.168.1.24/proxy/users/api/v2/info => [200]
13 [GET] https://192.168.1.24/proxy/users/api/v2/users/admin/uos?page_num=1&page_size=999 => [200]
14 [POST] https://192.168.1.24/api/controllers/checkUpdates => [204]
15 [GET] https://192.168.1.24/api/cloud/backup/settings/list => [200]
16 [GET] https://192.168.1.24/favicon.svg?v3 => [200]
17 [GET] https://192.168.1.24/favicon.ico?v3 => [200]
18 [GET] https://192.168.1.24/proxy/protect/api/bootstrap => [200]
19 [GET] https://192.168.1.24/proxy/protect/api/video-archive/fetch-pending => [200]
20 [GET] https://static.ui.com/fingerprint/ui/public.json => [200]
21 [GET] https://192.168.1.24/favicon.svg?v3 => [200]
22 [GET] https://192.168.1.24/favicon.ico?v3 => [200]
23 [PATCH] https://192.168.1.24/proxy/protect/api/users/6981595f01a13c03e41d8dec => [200]
24 [GET] https://192.168.1.24/proxy/protect/api/recognition/vehicle/groups?hasName=true&page=1&pageSize=1000000 => [200]
25 [GET] https://192.168.1.24/proxy/protect/api/recognition/face/groups?hasName=true&page=1&pageSize=1000000 => [200]
26 [GET] https://192.168.1.24/proxy/protect/api/arm/profiles => [200]
27 [GET] https://192.168.1.24/proxy/protect/api/weather => [200]
28 [GET] https://192.168.1.24/proxy/protect/api/active-sessions => [200]
29 [GET] https://192.168.1.24/proxy/protect/api/events?end=1770968198751&start=177088198751&types=aiprocessorTaskQueueInfoI
30 [GET] https://192.168.1.24/proxy/protect/api/events?limit=5&orderDirection=desc&types=smartDetectLine&types=smartDetectZi
31 [GET] https://192.168.1.24/proxy/protect/api/spotlights => [200]
32 [GET] https://192.168.1.24/proxy/protect/api/statistics/detections?end=1770968198752&period=3600000&start=1770883200000 :
33 [GET] https://static.ui.com/fingerprint/ui/uids/06d4d77c-3dd3-4691-8aac-3e1d0abb8699_icon => [200]
```

纪录 api 调用生成测试脚本

测试开启 protect 设定温度单位为华氏，然后跑 UI 的时候就观察 api 怎么 call 的，一起产出 api & ui 的测试脚本

Playwright Agents 不仅可用于开发 UI 自动化测试脚本，还能在自适应执行 UI 操作、观察用户真实业务流程的过程中，全程监听、捕获并记录所有网络请求与 API 调用详情，基于真实运行流量自动生成对应的接口测试脚本与断言逻辑，让 UI 自动化与 API 自动化形成天然联动，一次场景执行即可同步覆盖界面校验与接口逻辑验证，大幅提升测试资源利用率与场景覆盖度。



一次交互录制，双维度测试产出，实现 UI 与 API 自动化一举两得

基于 Skills 实现 JS 脚本到 Pytest 标准化迁移

```
---
name: playwright-js-to-pytest
description: Converts Playwright JavaScript/TypeScript test code to Playwright
pytest Python test code. Trigger this skill whenever the user provides a Playwright
JS/TS test file or code snippet and asks to convert it to Python, pytest, or
playwright-pytest format. Even if the user doesn't say "playwright-js-to-pytest"
explicitly, trigger this skill when they say things like "convert this playwright test to
python", "convert playwright JS to pytest", "I want to run these playwright tests in
python", or "playwright typescript to python".
---

# Playwright JS → Playwright pytest Converter
Your job is to accurately convert Playwright JavaScript/TypeScript test code into
Playwright pytest (Python) format. A good conversion is more than a syntax swap, it
produces idiomatic Python code that a Python developer would feel at home with,
while faithfully preserving the intent and logic of the original tests.

---

## Core Conversion Rules

### 1. Import Statements

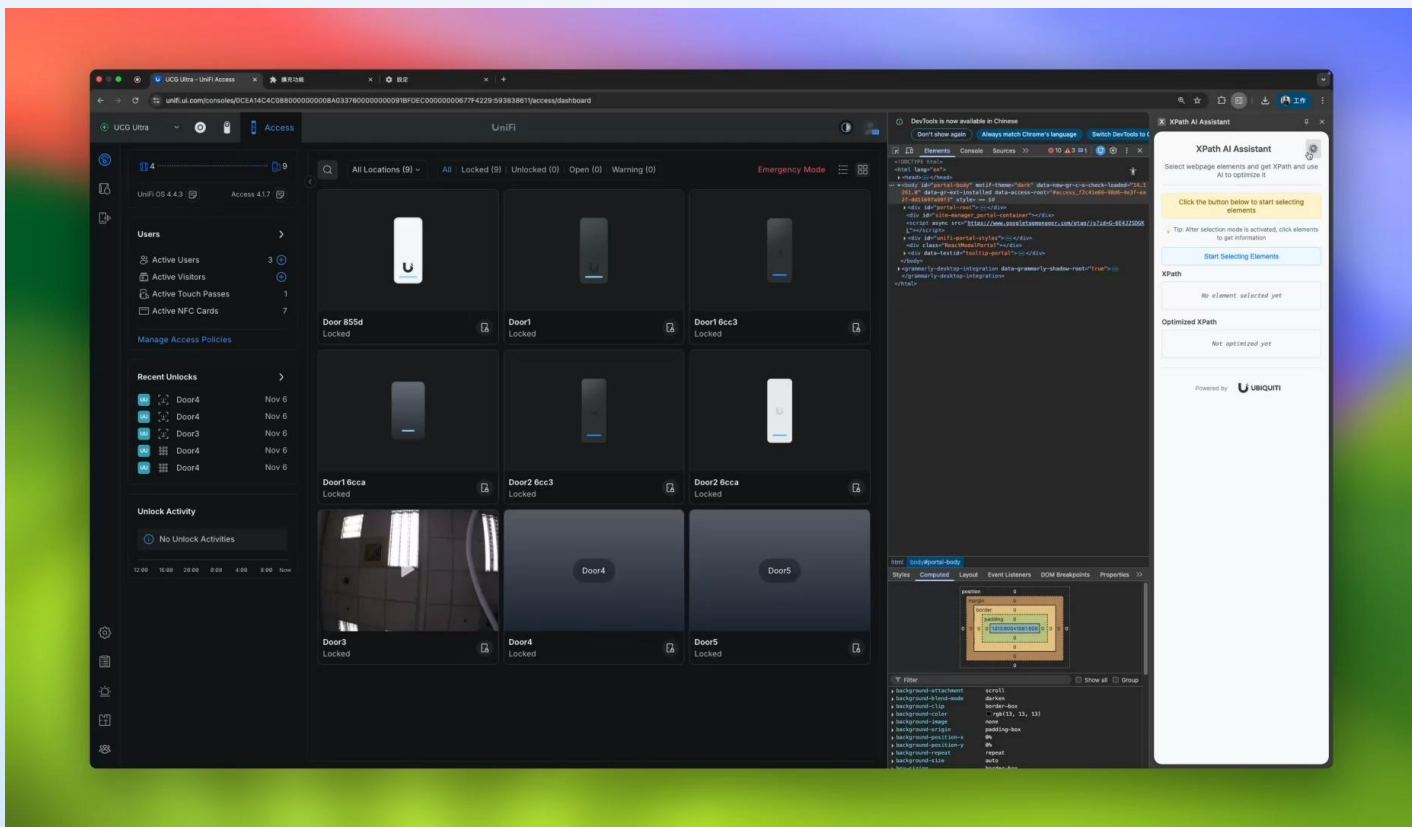
...

javascript// JavaScriptimport { test, expect } from '@playwright/test';
import { Page } from '@playwright/test';
.....
```

工具實踐部分 prompt

当前 Playwright Agent 原生仅支持 JavaScript 脚本生成，无法直接适配 Pytest 技术栈，通过构建专用转换 Skills 可实现无缝兼容：先由 AI 基于 Playwright Agent 生成标准 JS 测试脚本，再通过转换 Skills 完成语法、结构与断言的自动化迁移，最终输出可直接执行的 Pytest 用例，低成本实现跨语言测试自动化落地。

基于大语言模型，一键生成高稳定、易维护的 Xpath



工具操作画面

聚焦自动化测试痛点，AI 赋能 XPath 高效生成

针对传统方式生成 XPath 稳定性差、可读性低、需大量手动调整的痛点，推出基于大模型的 AI 定位小工具，实现一键提取与智能优化，显著降低脚本开发与维护成本。

AI 智能优化生成：依托大模型理解测试场景，生成精简、鲁棒、高可读的 XPath，避免冗余与兼容性问题，减少调试开销

自定义规则适配：支持自定义生成规则，优先选择带 id 元素，自动避开乱码 ID、随暗黑模式变动的 class 名，提升 XPath 稳定性与可维护性

基于大语言模型，一键生成高稳定、易维护的 Xpath

You are a UI automation test engineer tasked with writing XPath expressions. Your goal is to create an XPath that is suitable for automated testing based on the provided HTML elements.

You will be given three pieces of information:

1. The HTML of the current element
2. The current XPath of the element
3. The HTML of the parent node (up to three levels)

When generating the XPath, follow these rules:

1. Prioritize unique identifiers:
 - First choice: Use attributes like 'data-testid' if available
 - Second choice: Use parent containers with clear meanings (e.g., containing 'root', 'panel')
 - Use meaningful class names or text in the XPath
 2. Avoid classes with random characters:
 - Do not use classes that contain UUIDs or unclear business semantics
 3. Class name processing:
 - Filter out meaningless class names
 - Use 'contains()' or 'starts-with()' for partial matches
 4. Structural Stability:
 - Use relative paths such as `./div` and avoid absolute paths like `/html/body`.
 - Generated XPath should be robust and not require describing the entire HTML structure layer by layer.
-

工具实践部分 prompt

工具	結果
Playwright	<code>page.locator("li").filter(has_text="Interface Designer").locator("a").click()</code>
Cheome	<code>//*[@id="access_fc0ff1db-77f8-47fe-b89a-6105c113fd3d"]/div/div[1]/div[1]/nav/div[1]/ul/li[3]/div/a</code>
3rd party	<code>(//a[@class='component__rHEvxowz icon-dark__rHEvxowz is-accessible__rHEvxowz'])[2]</code>
自研工具	<code>//li[contains(@class,'auto-interface-designer')]/a[contains(@class,'component')]</code>

在透过 JS 获取目标元件 DOM 后，我们采用向上回溯三层父节点的策略为 AI 提供上下文依据；然而，面对 SVG 等庞大结构时，为避免 Token 浪费，系统会预先执行过滤机制，剔除对大模型推理无效的冗余数据，仅保留关键特征以优化上下文效率。

测试稳定性与执行效率优化

善用 IDE Prompt 功能技巧

善用 Cursor 内置 Prompt 工具，通过 @文件引用与 /Command 指令强化上下文输入，可在人机协作中引导 AI 快速聚焦、收敛问题，有效避免思路发散，提升交互精准度。

人为介入特殊行为

在 AI 执行自动化操作时，浮动的 Toast 讯息常会遮挡元素导致点击失效。建议将其视为环境前置处理，比照 Fixture 的逻辑，在测试开始前透过脚本将其移除或停用 pointer-events，避免 AI 在探测路径时因遮挡而报错。



高效精准人机交互

IDE Agent Mode 强化协作

善用开发工具中的 Agent Mode 可以突破单纯的代码补全限制。在此模式下，AI 具备更强的专案感知与自主执行能力，能跨档案理解测试架构并自动修正错误，将 AI 从「助手」提升为能独立解决问题的「代理人」。

模型能力决定测试成效

测试生成的稳定性高度依赖模型的推理能力。模型愈强，对复杂逻辑与异常流程的处理能力就愈高；不管是测试规划或是对当前框架架构的理解，虽然高阶模型运算成本较高，但其带来的高成功率能显著减少人工回头 Debug 的成本。

AI 赋能测试革新：从点效突破到全域提速

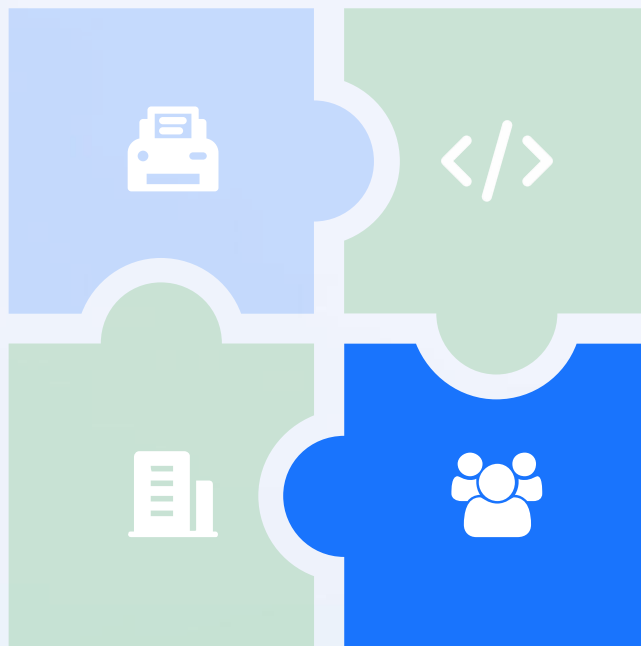
以智能技术重构测试全流程，实现从单点优化到全链路提效的跨越

测试用例生成

基于 AI 的分析能力，快速解析需求文档与接口定义，透过结构提示词，精准生成测试用例，大幅缩短用例设计周期，摆脱人工编写的低效重复。

产品业务知识提取

通过大模型处理与知识图谱技术，从产品文档、历史项目中高效提取核心业务规则与逻辑关联，快速构建测试知识底座，减少业务理解成本及作为测试案例设计参考文档。



测试脚本生成

AI Agent 自主理解测试场景，自动生成可执行脚本并完成自动化测试执行，支持多场景适配，降低脚本编写门槛，提升测试执行效率与稳定性。

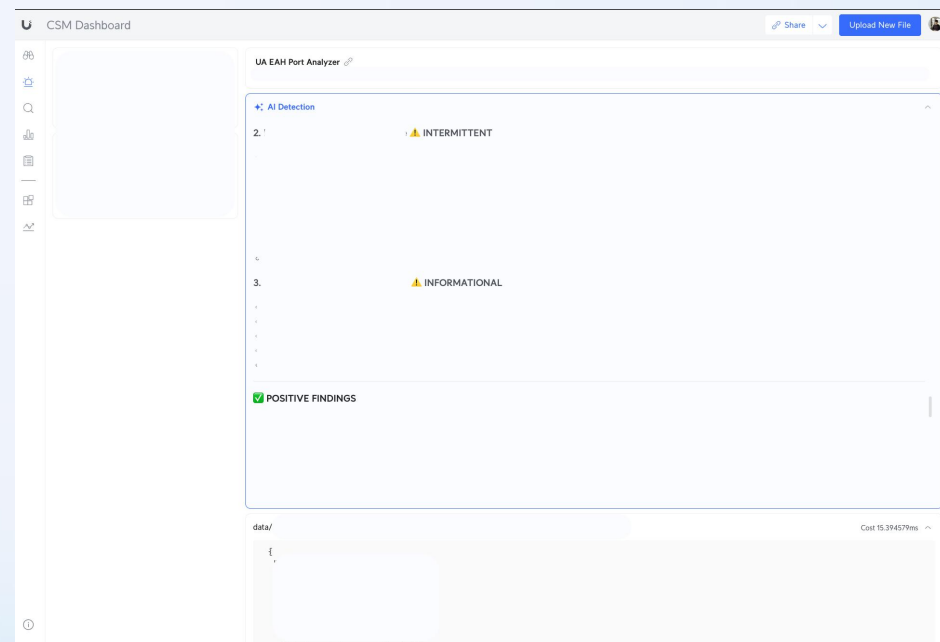
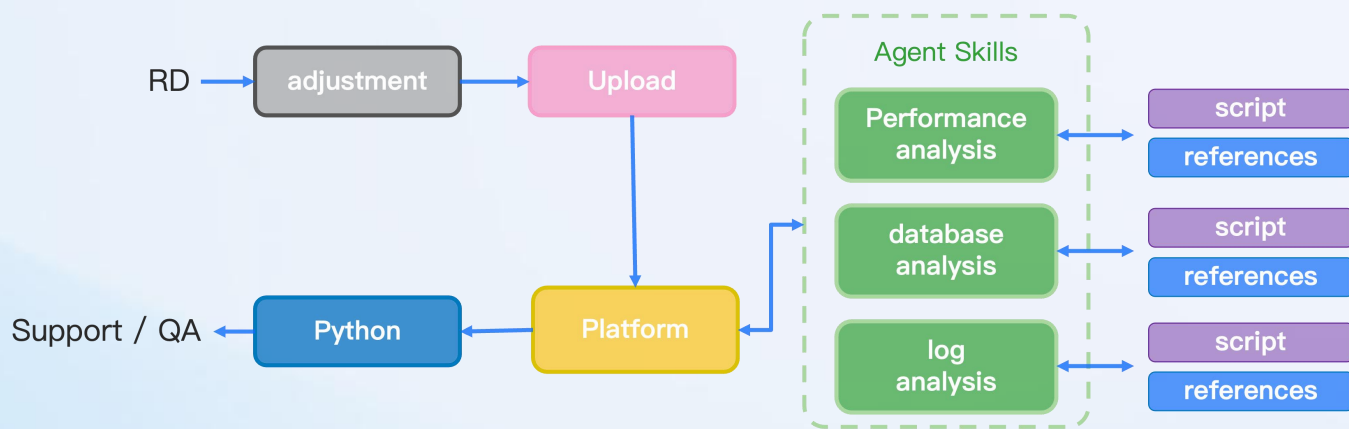
用户反馈 Log 分析

AI 智能解析海量用户反馈与日志，自动定位高频问题、异常场景及潜在风险点，将零散数据转化为可落地的测试重点。

AI 驱动 Log 分析与 Skills 从人工排查到自动诊断

通过 AI 辅助完成用户日志的智能诊断与问题定位，构建人工协同排查→经验沉淀→自动分析的完整闭环流程，同时通过日志前置过滤实现 Token 成本与分析效率的双重优化。

- **第一阶段，资产沉淀：**RD 通过与 AI 对话式协同排障，将零散的专家经验标准化封装为可复用的 Skills，构建组织级知识库。
- **第二阶段，自动闭环：**新日志上传即自动调用 Skills 进行智能预判；配合 Python 前置过滤引擎清洗噪声，在大幅降低 Token 成本的同时，实现从「人工排查」到「自动诊断」的效率跃升。

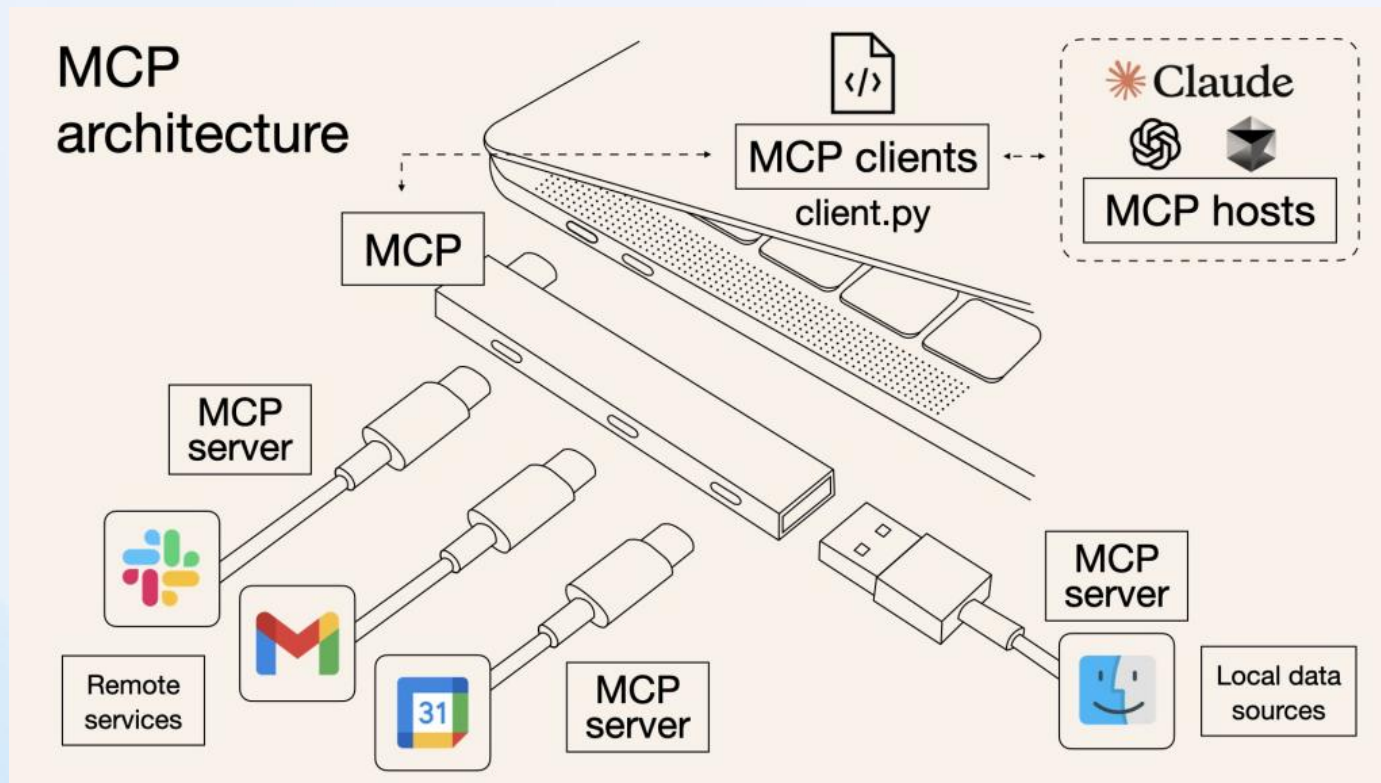


03

核心技术底座

Agent Skills + Playwright Agent + MCP 的协同架构

● 關於 MCP (Model Context Protocol)



Model Context Protocol (MCP) 是一种用来让大型语言模型与外部工具、资料来源和系统安全互动的开放协议。它提供标准化的方式，让模型可以像使用「外挂」一样连接资料库、API、档案系统或各种应用程式，并在需要时取得即时资讯或执行操作。透过MCP，开发者不必为每个工具单独整合，大幅降低整合成本，同时也能让AI系统更可扩展、可控。

ANTHROPIC

MCP connects Claude to external services and data sources. Skills provide procedural knowledge —instructions for how to complete specific tasks or workflows.

关于 Agent Skills

当前的大模型已经具备非常强的通用能力，但是专业技能才是解决特定领域问题的关键。AI 必须透过专业领域知识克服『懂而不精』的局限，而 Agent Skills 的诞生就是通过赋予 AI Agent 专业技能与知识，让 AI Agent 能够更有效完成任务。

2025/10

Agent Skills 诞生

推出以文件夹为核心的技能机制，开启 AI 任务模组化时代，让大模型能自主读取特定文件夹中的指令与脚本，学会操作 PDF、Slack 等特定工具。

2025/12

社群及 OpenAI 加入

在 Agent Skills 发布后不久，开源生态系，涌现大量相关开源专案，OpenAI 将 Agent Skills 整合进 ChatGPT 与 Codex 体系，验证了 Skills 框架的可扩展性。

2025/12

Anthropic 推出标准

Anthropic 进一步发布跨平台标准化 Agent Skills，旨在解决不同 AI 系统间的相容性问题，推动 Agent 像软体 API 一样具备通用性与互通性。

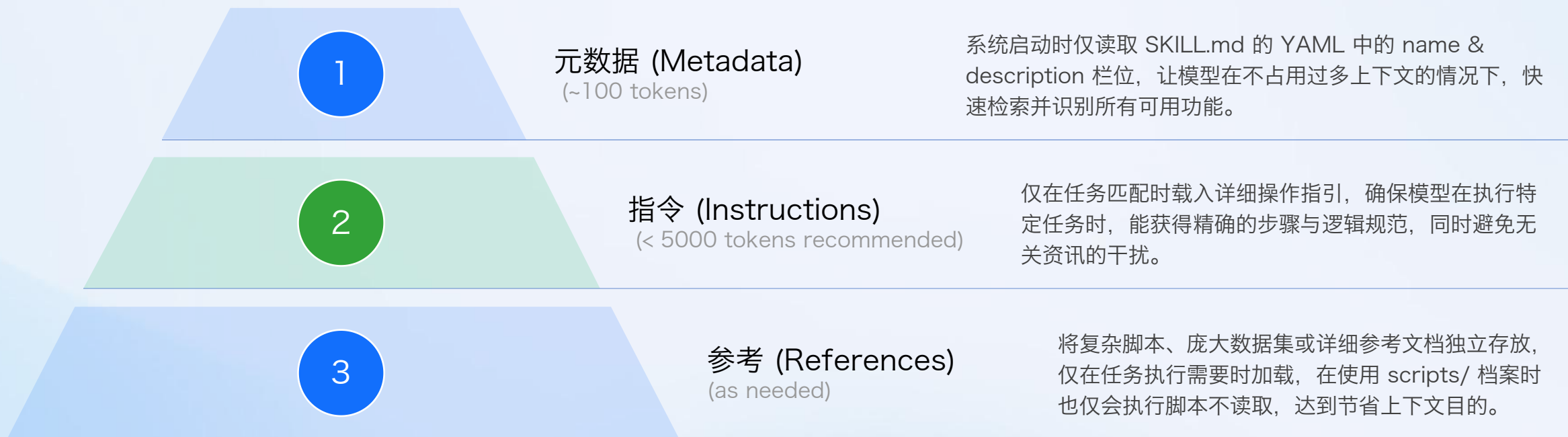
2026/01

Agent Skills 时代

在 Anthropic 推出标准化框架后，Cursor、GitHub Copilot 与扣子纷纷将 Agent Skills 整合进其生态系。各大平台的强力支持，促使 Agent Skills 实现了跨平台的互通互操作。

Agent Skills 渐进式揭露：实现上下文的效能最大化

Agent Skills 的核心创新在于「渐进式揭露」机制，将技能资讯结构化三个层次并采按需逐步载入，在确保任务细节完整性的同时，有效避免一次性过度占用上下文窗口。



Agent Skills 专案架构与文件规范

pict-case/SKILL.md

```
---
name: pict-case
description: Design test cases using PICT
(Pairwise Independent Combinatorial
Testing).
---

# PICT Test Designer
Use PICT to design test cases from
requirements or code

Model structure:
- Syntax and constraint grammar: See
[references/pict_syntax.md](references/
pict_syntax.md)

### 3. Execute model and get test cases
- Or use Python to build the model (and
optionally run via pypict: pip install pypict)
See
[scripts/pict_helper.py](scripts/pict_helper.p
y)
```

pict-case/references/pict_syntax.md

```
# PICT Syntax Reference

## Model file structure

1. Parameter definitions (one per line)
2. Sub-model definitions (optional)
3. Constraint definitions (optional)

## Parameter definitions
```

pict-case/scripts/pict.py

```
import json
import sys
from pathlib import Path

def generate_model(config_path: str) ->
str:
    """Read JSON config and return PICT
    model text."""
    with open(config_path) as f:
        config = json.load(f)
```

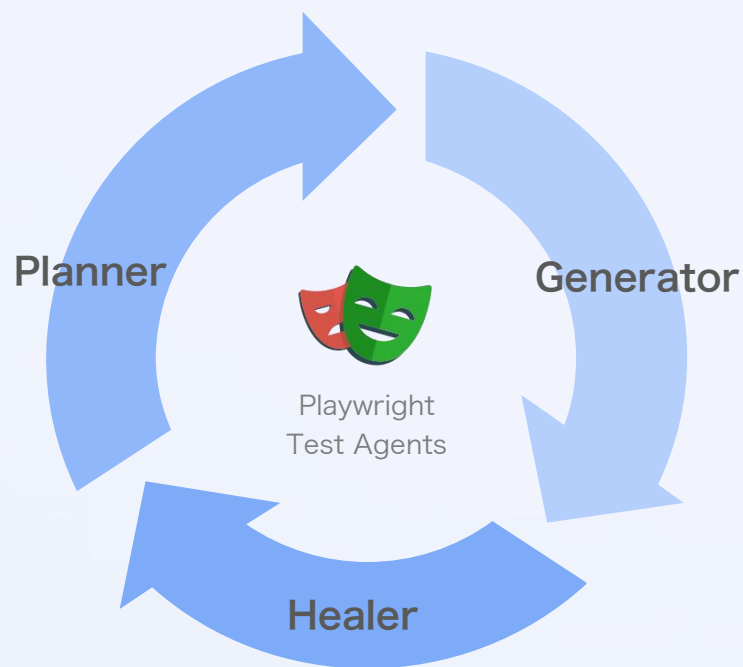
Agent Skills 作为上下文工程核心方案，其专案架构以标准化文件为核心载体，整体遵循插件化目录设计逻辑，核心文件与结构化内容构成技能落地的关键，同时明确的文件规范让技能具备高复用性与场景适配性，解决了通用 Prompt 碎片化、复用性低的问题。

```
my-skill/
├── SKILL.md      # required: instructions + metadata
├── scripts/      # optional: executable code
├── references/   # optional: documentation
└── assets/       # optional: templates, resources
```

什么是 Playwright Agents

Playwright Test Agents 是一种以 AI 为核心的测试自动化机制，能自主规划 (Planner)、生成 (Generator) 与修复 (Healer) Playwright 端对端测试流程，从而提升测试覆盖率与维护效率。

Planner (规划者)： 意图转化与自动化导航
将自然语言描述的测试目标（如：输入帐号后登入成功）转化为结构化的 Markdown 测试计画，透过自动探索应用程序路径，确保测试覆盖逻辑精准且易于人类审阅。



Generator (生成者)： 撰写脚本自动生成
根据测试计画自动撰写 Playwright 脚本，并在生成过程中即时对照真实 UI 验证选择器 (Selectors) 与断言，确保产出的程式码具备高可靠性与立即执行力。

Healer (修复者)： 自我修复与降低维护成本

监控测试执行状态，当 UI 变动导致失败时，自动分析 DOM 结构并修复定位器或同步逻辑，透过自动重跑验证修复结果，大幅减少人工维护脚本的负担。

Playwright Agents 核心依赖：两类 MCP Server 定位与差异

	playwright - mcp	playwright - test - mcp
核心定位	通用浏览器自动化 MCP	测试专属、面向 Playwright Test 的 MCP Server
核心定位	让 AI 操控浏览器、操作页面	支撑 AI 生成、运行、修复 Playwright 测试脚本
面向场景	网页操作、爬虫、自动化交互	E2E 测试开发、CI 故障修复、测试脚本维护
AI 能力范围	browser_click、browser_navigate、browser_fill 等页面操作	test_run、test_list、planner、generator、healer 等测试能力
源代码	单独仓库： microsoft/playwright-mcp	内置于 Playwright 主仓库

playwright - mcp 与 playwright - test - mcp 同为微软推出的 Playwright MCP 工具，二者名称易混淆，但定位、能力、场景完全不同。

playwright - mcp 面向通用浏览器自动化，让 AI 直接操控页面；

playwright - test - mcp - server 面向测试工程，支撑 AI 完成测试编写、执行与自修复，是本次 Playwright Agent 的核心底座。



AOM (Accessibility Object Model) 自动化测试的语义化革新

```
讓我先了解現有的測試結構和種子檔案。
Explored 2 files 3 searches >
好的，我來幫你建立這個測試。首先讓我設定頁面並逐步執行測試流程。
讓我先用種子檔案設定頁面，然後逐步操作。

✓ Ran generator_setup_page:cursor-playwright-playwright-test
RESPONSE
Running 1 test using 1 worker
Paused at end of test. ready for interaction
Page state
• Page URL: https://192.168.1.24/protect/dashboard
• Page Title: Dashboard | Network Video Recorder Pro - Max
• Page Snapshot:
  - generic [ref=e2]:
  - generic [ref=e3]:
  - banner [ref=e4]:
    - button "Site Switcher":
      - generic:
        - generic:
          - img
        - generic "Network Video Recorder Pro - Max"
    - generic [ref=e5]:
    - link "Protect" [ref=e6] [cursor=pointer]:
      - /url: /protect/dashboard
    - generic [ref=e7]:

Thought for 2s
頁面已登入並在 Dashboard 上。現在我逐步執行測試流程。
> 2 Files Review
Plan, @ for context, / for commands
Agent: claude-4.6-opus-high
```

实际 AOM 架构图

AOM 作为网页无障碍访问的核心技术规范，通过 JavaScript API 直接操作页面无障碍对象模型，为自动化测试提供了超越 XPath/CSS 的定位与交互方案。其核心价值在于将元素定位从“视觉属性依赖”升级为“语义逻辑关联”，为自动化测试在元件的选择上开创了更多的可能：

- 1 **语义化定位更稳定：**直接基于元素角色（role）、标签（label）等无障碍语义属性定位，不受 DOM 结构、class/id 变动影响，从根源解决传统定位脆弱性问题
- 2 **与 AI 协同提效：**AOM 将复杂 DOM 转化为标准语义树，消除无效干扰，使 AI 能够精准识别业务逻辑，大幅提升自动化脚本的生成成功率。
- 3 **摆脱 ID 依赖的关联定位：**支援直接建立元素间的无障碍关联（如 aria-controls），无需为成千上万的动态元素生成唯一 ID，大幅简化复杂 UI（如无限滚动列表）的测试逻辑与维护成本。

● 分层协同的 Skills + MCP 混合架构



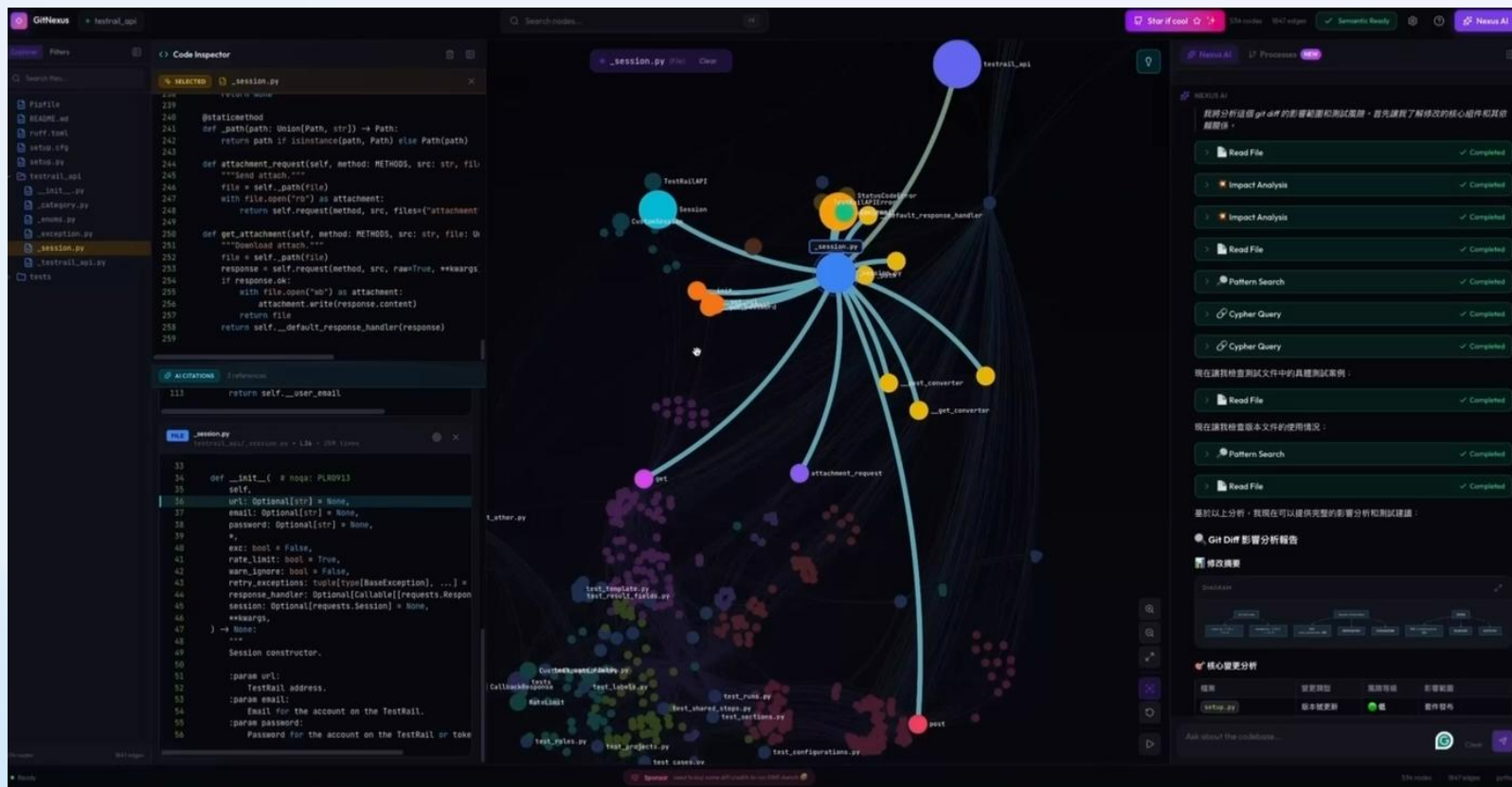
Skills 与 MCP 并非替代关系，而是形成分层协作的混合架构，通过关注点分离实现能力与智慧的互补，构建高效、可维护的智能体执行体系，在实际工作流程中完成从任务识别到结果输出的全链路闭环，同时实现成本优化与能力复用的双重价值。

04 未來展望

Agent Skills + Playwright Agents + MCP 的协同架构



基于代码依赖分析实现测试范围精准聚焦



GitNexus 开源工具

将代码库构建为全量知识图谱，可精准解析代码修改内容与模块间的依赖关系及调用链，让测试工作从全量覆盖转向基于变更风险的精准聚焦。借助图谱变更影响检测能力，可量化代码修改的风险，识别高风险依赖模块，让测试资源集中投入到风险高发的变更关联区域，实现测试效率与测试深度的双重提升。

- **精准识别：**通过图谱解析代码修改点的上下游依赖系，明确变更影响边界
- **风险量化：**对修改影响进行深度分级与置信度评估，标记高风险的核心模块与执行流程
- **精准测试：**基于风险等级规划测试范围，聚焦高风险变更关联区域，减少无意义的全量测试

构建预先冒烟机制，实现 AI 自动生成、执行与代码化回归

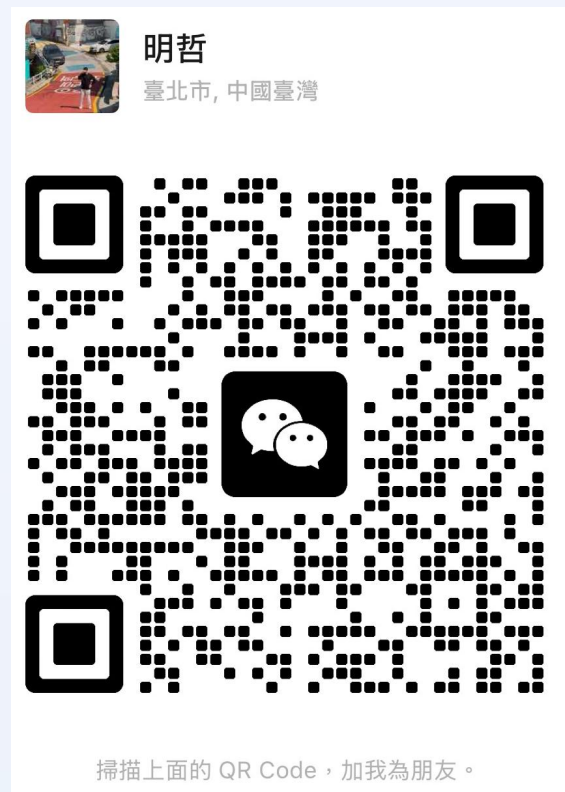
开发提交 PR 后，调度 AI 基于代码变更精准生成用例并执行冒烟测试，生成冒烟测试案例；验证通过后，AI 自动将测试逻辑转化为标准自动化脚本反馈至仓库，同步推送结构化报告给 QA，实现「测试左移」与回归资产的自动沉淀，让 QA 专注于高价值场景复核。





AI 对 QA 是帮助比较多？
还是威胁比较多？

欢迎交流



THANKS

大模型正在重新定义软件

Large Language Model Is Redefining The
Software



AiCon

全球人工智能开发与应用大会

上海站

探索 AI 应用边界

Explore the limits of AI applications

2026 年 6 月 26-27 日 / 上海张江科学会堂



参会咨询

专题：世界模型与多模态智能突破

专题出品人：朱政

极佳视界 / 联合创始人 & 首席科学家



专题：Agent 系统架构与工程化实践

专题出品人：鲁浩楠

OPPO / 大模型算法负责人



专题：AI 原生数据工程

专题出品人：王彦辉

火山引擎 / 数智平台产品总监



专题：Agent 安全、评测与可信治理

专题出品人：马传雷（岳立）

支付宝 / 业务风险技术部负责人



专题：Agent 数据、记忆与运行时基础设施

专题出品人：邓亚峰

EverMind / CEO



专题：企业级研发体系的重构

专题出品人：沈浪

快手 / 研发效能负责人

